

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL



**FACULTAD DE INGENIERÍA EN
SISTEMAS, ELECTRÓNICA E INDUSTRIAL**

CARRERA DE SOFTWARE

Título: Trabajo en equipo y colaboración en GitHub – Tower Defense

Integrante que responde:

Chalco Tasna Kenneth Mateo

Integrantes:

Camacho Monta Josue Jampier
Chalco Tasna Kenneth Mateo
Cunalata Mendoza Damian Alexander
Silva Camuendo Luis Alexander
Tacuri Santillan Monica Sara
Tisalema Guashco Darwin Joel

Ciclo Académico y Paralelo: Tercero “B”

Módulo y Docente: Estructura de Datos – Ing. José Caiza.

JULIO 2026 - DICIEMBRE 2026

Trabajo en equipo y colaboración en GitHub

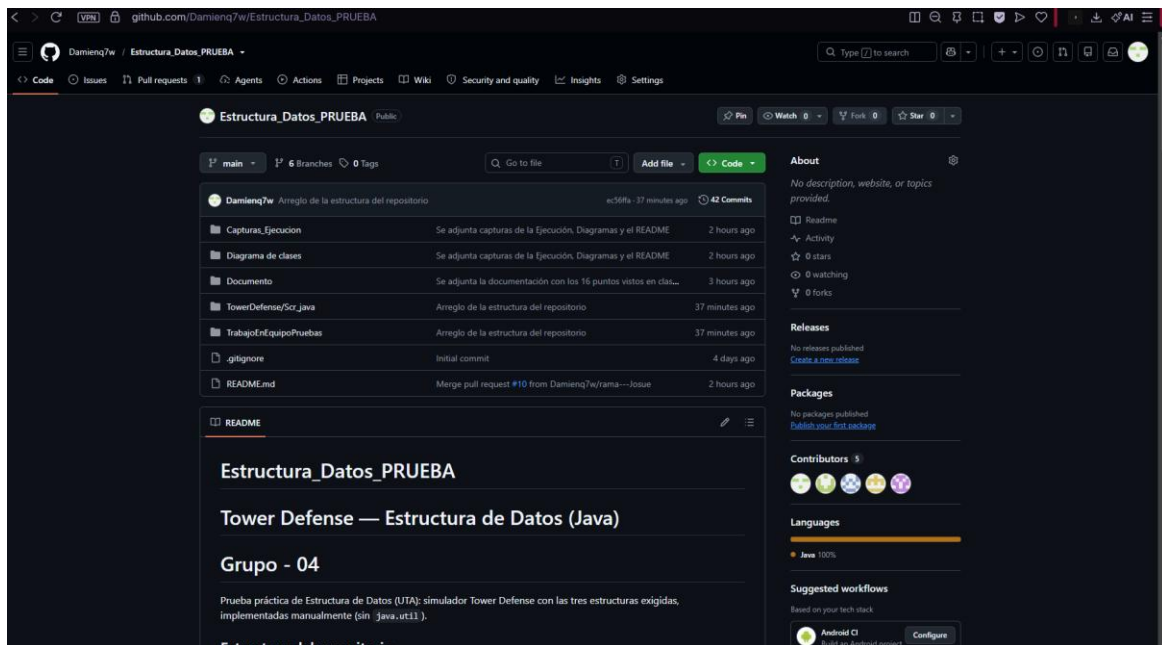
Integrante que responde: Chalco Tasna Kenneth Mateo

Presento a continuación las evidencias verificables de cómo nuestro grupo organizó y desarrolló el proyecto Tower Defense mediante GitHub, identificando claramente mi propia contribución y la de mis compañeros.

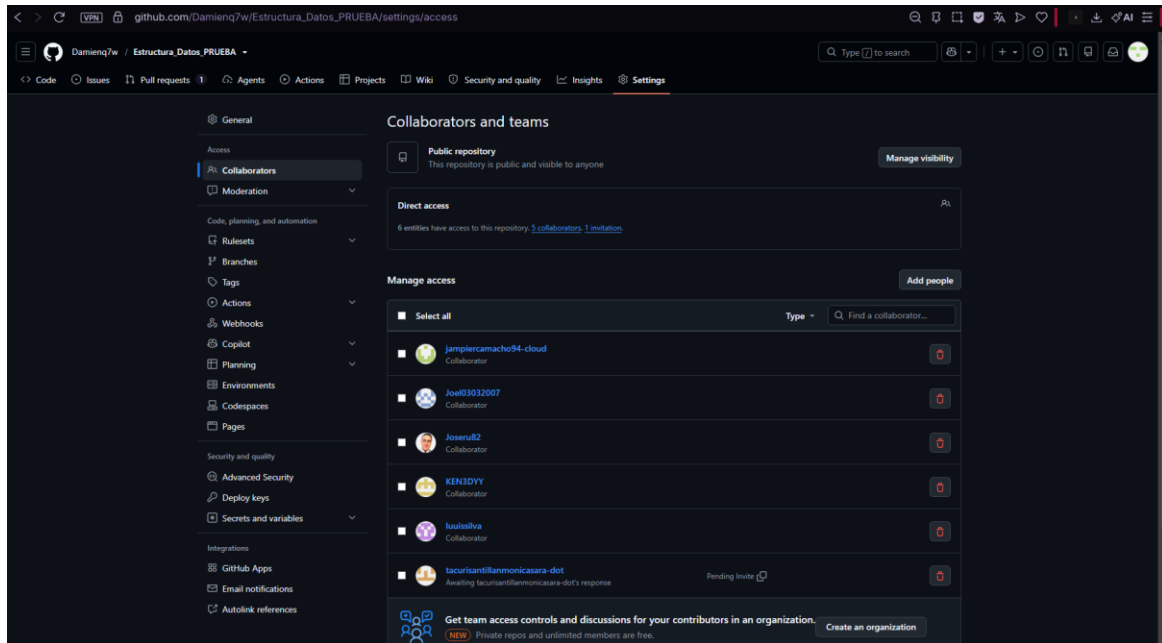
1. Enlace al repositorio

https://github.com/Damieng7w/Estructura_Datos_PRUEBA

El docente (Ing. José Caiza) ha sido invitado como colaborador del repositorio, con acceso habilitado para revisar el código, los commits, las issues, los Pull Requests y el historial completo.



Captura de la página principal del repositorio en GitHub.



Captura de Settings → Collaborators, donde consta el docente como colaborador del repositorio.

2. Integrantes, función asumida y rama utilizada

Integrante	Función asumida	Rama
Cunalata Mendoza Damian Alexander	Líder técnico e integración — Torre, TowerDefenseApp y TowerDefenseGUI. <i>Desarrolló Torre y TowerDefenseApp (menú de consola, integración de las tres estructuras exigidas y lógica de “avanzar turno”), además de TowerDefenseGUI (interfaz gráfica opcional). Realizó la compilación final y la prueba del caso sugerido.</i>	Damian_Cunalata
Tisalema Guashco Darwin Joel	Desarrollo — ListaSecuencialTorres, Main y NodoEnemigo. <i>Creó NodoEnemigo, NodoOleada y Oleada, y la clase ListaSecuencialTorres completa (arreglo + contador, con insertar, eliminar por id, buscar, mostrar y contar torres activas).</i>	Rama-Joel
Silva Camuendo Luis Alexander	Desarrollo — NodoOleada, Oleada y TableroPanel. <i>Implementó ListaDobleEnemigos y dejó lista la base de ListaSecuencialTorres, además de TableroPanel, el panel de Swing que dibuja el camino, las torres, los enemigos y los efectos de combate de la interfaz gráfica opcional.</i>	Rama-Luis-Silva
Chalco Tasna Kenneth Mateo	Desarrollo — Enemigo, Jugador, ListaCircularOleadas y ListaDobleEnemigos. <i>Creó Enemigo (avanzar, recibirDanio, estaDestruído) y Jugador (vidas, derrota), implementó ListaCircularOleadas (registrar, avanzar y reiniciar el ciclo de</i>	Mateo-Chalco

	<i>oleadas) y ListaDobleEnemigos (inserción, búsqueda por id, eliminación y recorrido bidireccional).</i>	
Camacho Monta Josue Jampier	Documentación — README.md, diagrama de clases y capturas de ejecución. <i>Elaboró y mantuvo el README.md (estructura del proyecto, explicación de cada estructura de datos, diagrama de clases) y las capturas de pantalla de la ejecución del sistema.</i>	rama---Josue
Tacuri Santillan Monica Sara	Desarrollo (no completado) — Validaciones de ListaSecuencialTorres (lista vacía, llena, id repetido). <i>Tenía asignadas las validaciones de ListaSecuencialTorres, pero no llegó a tiempo a la prueba práctica y no realizó su parte, por lo que no registra commits en el repositorio.</i>	—

3. Commits significativos (al menos 2 por integrante)

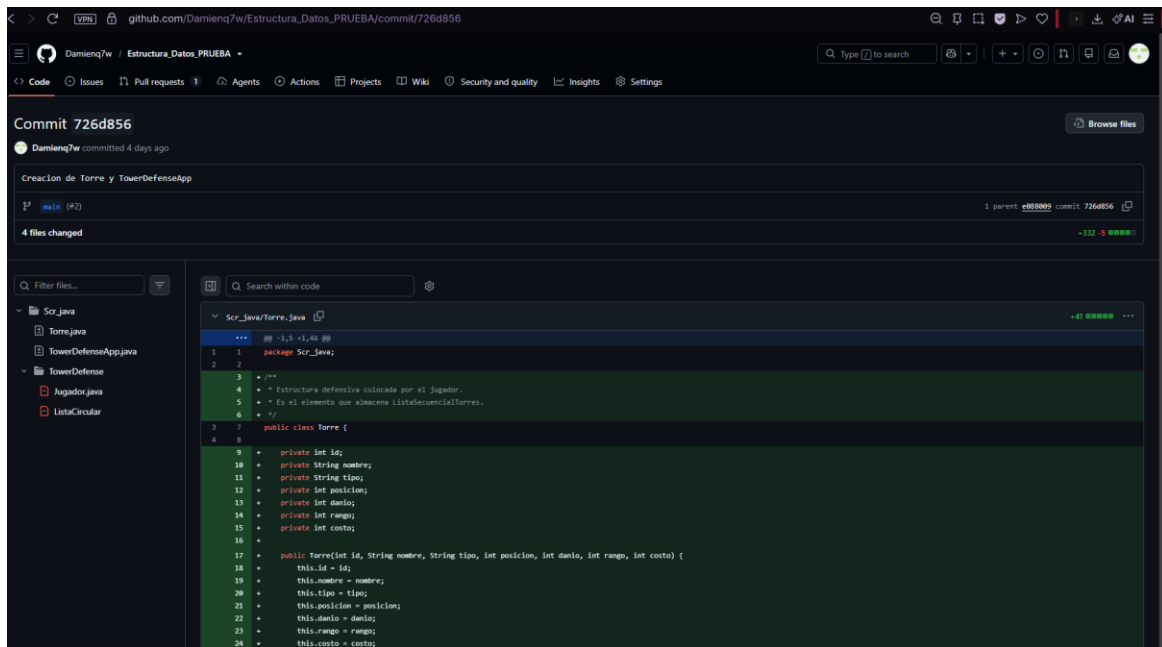
Cunalata Mendoza Damian Alexander

https://github.com/Damienq7w/Estructura_Datos_PRUEBA/commit/726d856

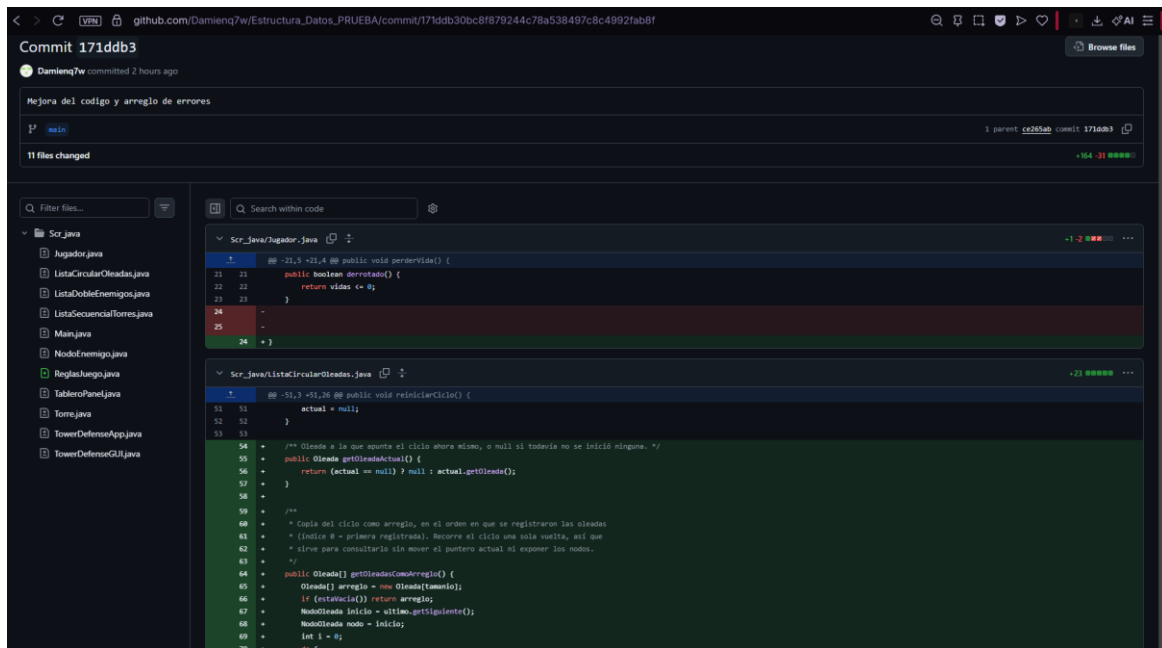
726d856 — "**Creacion de Torre y TowerDefenseApp**". Creó Torre.java y TowerDefenseApp.java, integrando la lista secuencial de torres, la lista doble de enemigos y la lista circular de oleadas dentro del menú de consola y la lógica de "avanzar turno".

https://github.com/Damienq7w/Estructura_Datos_PRUEBA/commit/171ddb3

171ddb3 — "**Mejora del codigo y arreglo de errores**". Agregó getOleadaActual() y getOleadasComoArreglo() a ListaCircularOleadas, creó ReglasJuego.java para centralizar las reglas compartidas entre la consola y la interfaz gráfica, y aplicó ajustes en Jugador, Main, NodoEnemigo, TableroPanel, Torre y TowerDefenseApp (11 archivos, +164/-31).



Captura del commit 726d856 en GitHub.



Captura del commit 171ddb3 en GitHub.

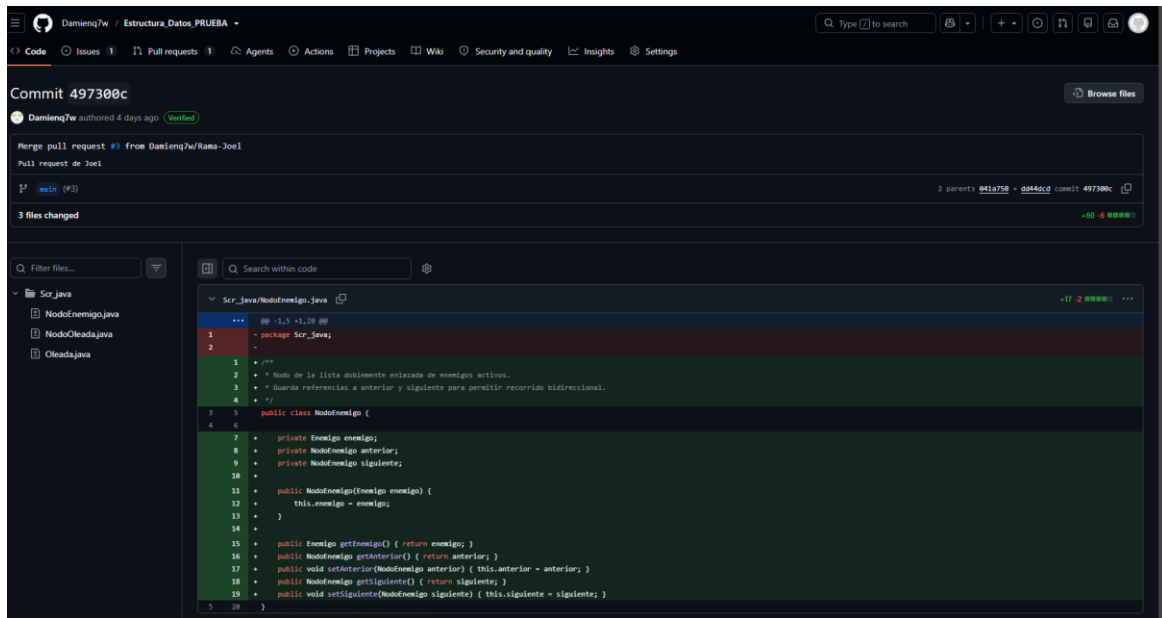
Tisalema Guashco Darwin Joel

https://github.com/Damienq7w/Estructura_Datos_PRUEBA/commit/dd44dcd

dd44dcd — "Creacion de las clases **NodoEnemigo**, **NodoOleada** y **Oleada**". Creó desde cero las tres clases: el nodo doble de enemigos, el nodo circular de oleadas y la plantilla de una oleada (cantidad, tipo, vida base y velocidad base).

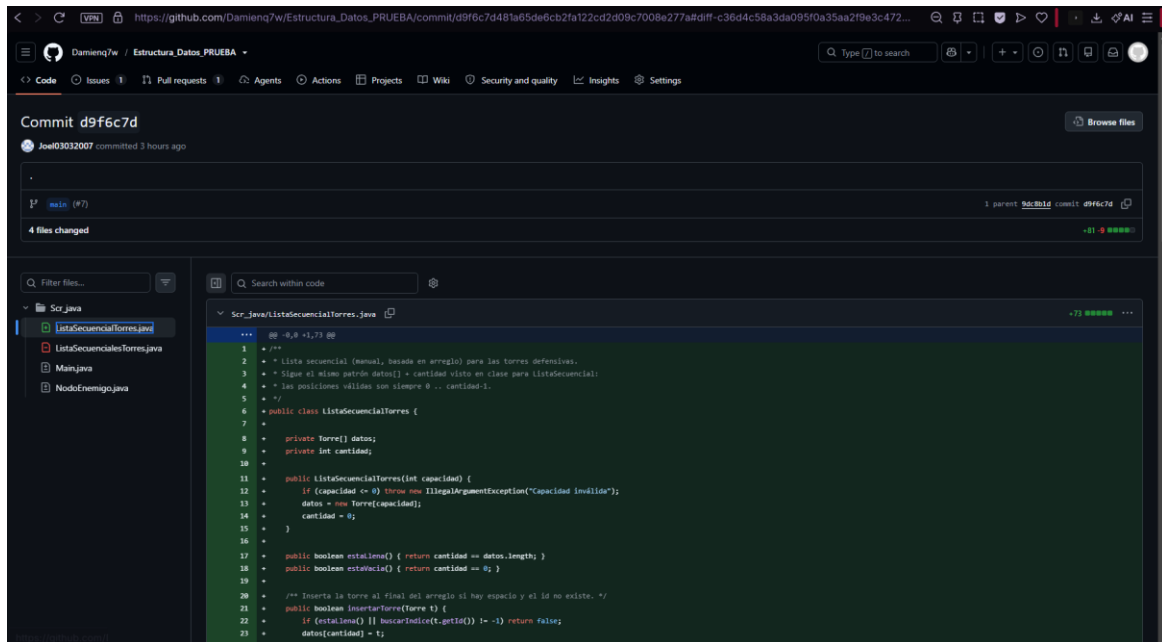
https://github.com/Damienq7w/Estructura_Datos_PRUEBA/commit/d9f6c7d

d9f6c7d — "Creación de ListaSecuencialTorres". Creó ListaSecuencialTorres.java completa (73 líneas: arreglo Torre[] datos + contador, insertar, eliminar por id, buscar, mostrar y contar torres activas), eliminó el archivo duplicado con el nombre mal escrito (ListaSecuencialesTorres.java) y ajustó Main.java y NodoEnemigo.java.



```
*** @@ -1,5 +1,20 @@
1 - package Scr_java;
2 -
3 + /**
4 +  * Nodo de la lista doblemente enlazada de enemigo activo.
5 +  * Guarda referencias a anterior y siguiente para permitir recorrido bidireccional.
6 +  */
7
8 + public class NodoEnemigo {
9 +
10 +     private Enemigo enemigo;
11 +     private NodoEnemigo anterior;
12 +     private NodoEnemigo siguiente;
13 +
14 +     public NodoEnemigo(Enemigo enemigo) {
15 +         this.enemigo = enemigo;
16 +     }
17 +
18 +     public Enemigo getEnemigo() { return enemigo; }
19 +     public NodoEnemigo getAnterior() { return anterior; }
20 +     public void setAnterior(NodoEnemigo anterior) { this.anterior = anterior; }
21 +     public NodoEnemigo getSiguiente() { return siguiente; }
22 +     public void setSiguiente(NodoEnemigo siguiente) { this.siguiente = siguiente; }
23 + }
```

Captura del commit dd44dcd (fusionado como 497300c) en GitHub.



```
*** @@ -0,0 +1,73 @@
1 + /**
2 +  * Lista secuencial (manual, basada en arreglo) para las torres defensivas.
3 +  * Sigue el mismo patrón datos[] + cantidad visto en clase para ListaSecuencial.
4 +  * Las posiciones válidas son siempre 0 ... cantidad-1.
5 +  */
6 + public class ListaSecuencialTorres {
7 +
8 +     private Torre[] datos;
9 +     private int cantidad;
10 +
11 +     public ListaSecuencialTorres(int capacidad) {
12 +         if (capacidad <= 0) throw new IllegalArgumentException("Capacidad inválida");
13 +         datos = new Torre[capacidad];
14 +         cantidad = 0;
15 +     }
16 +
17 +     public boolean estaLlena() { return cantidad == datos.length; }
18 +     public boolean estaVacía() { return cantidad == 0; }
19 +
20 +     /** Inserta la torre al final del arreglo si hay espacio y el id no existe. */
21 +     public boolean insertarTorre(Torre t) {
22 +         if (estaLlena() || buscarIndice(t.getId()) != -1) return false;
23 +         datos[cantidad] = t;
24 +         cantidad++;
25 +     }
26 + }
```

Captura del commit d9f6c7d en GitHub.

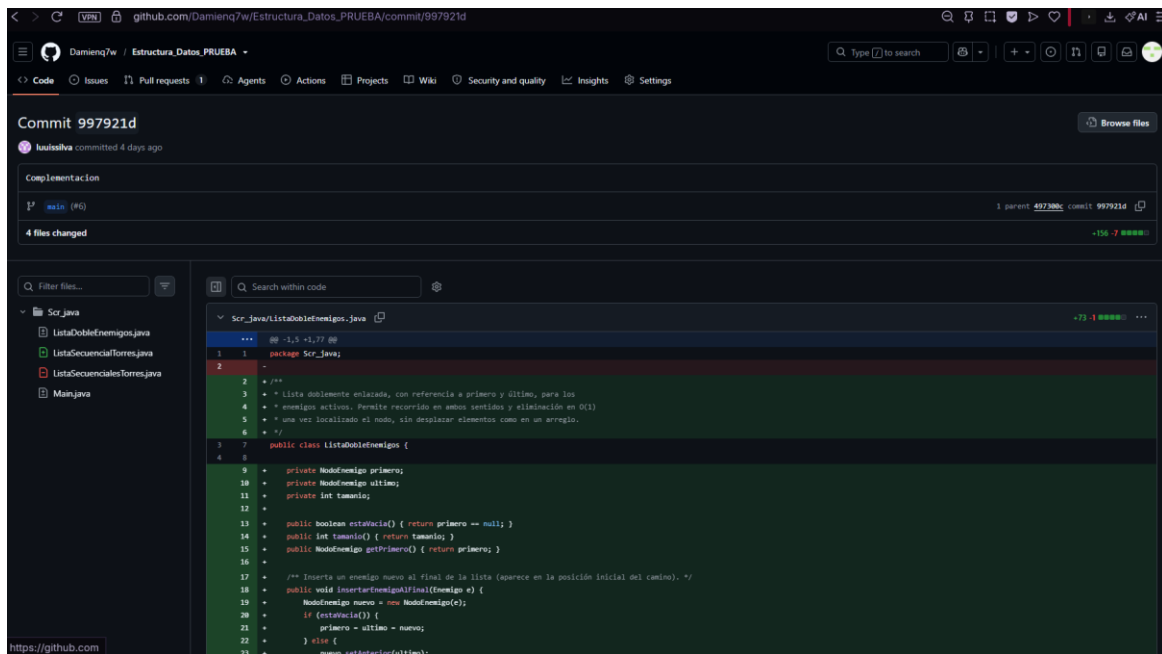
Silva Camuendo Luis Alexander

https://github.com/Damienq7w/Estructura_Datos_PRUEBA/commit/997921d

997921d — "Complementacion". Implementó ListaDobleEnemigos.java (inserción, eliminación y recorrido bidireccional) y ListaSecuencialTorres.java, y actualizó Main.java para delegar a TowerDefenseApp.main().

https://github.com/Damienq7w/Estructura_Datos_PRUEBA/commit/62f58a4

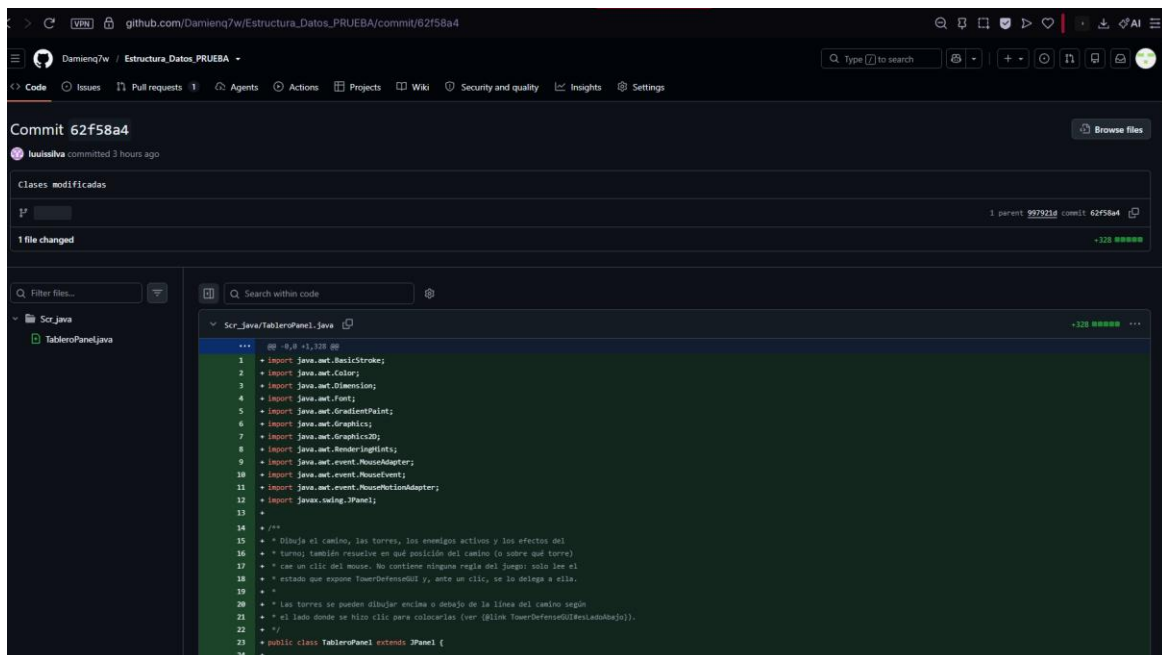
62f58a4 — "Clases modificadas". Creó TableroPanel.java (328 líneas): el panel de Swing que dibuja el camino, las torres, los enemigos, las barras de vida y los efectos de combate de la interfaz gráfica opcional.



The screenshot shows the GitHub interface for commit 997921d. The file list on the left includes Scr.java, ListaDobleEnemigos.java, ListaSecuenciaTorres.java, and Main.java. The main area displays the code for Scr.java/ListadoEnemigos.java, which is a Java class for managing a list of enemies. The code includes package declarations, imports, and methods for adding and removing enemies from the list.

```
1 package Scr.java;
2
3 /**
4  * Lista doblemente enlazada, con referencia a primero y último, para los
5  * enemigos activos. Permite recorrerla en ambos sentidos y eliminación en O(1)
6  * una vez localizado el nodo, sin desplazar elementos como en un arreglo.
7  */
8 public class ListadoEnemigos {
9
10     + private NodeEnemigo primero;
11     + private NodeEnemigo ultimo;
12     + private int tamaño;
13
14     + public boolean estaVacia() { return primero == null; }
15     + public int tamaño() { return tamaño; }
16     + public NodeEnemigo getPrimero() { return primero; }
17
18     /** Inserta un enemigo nuevo al final de la lista (aparece en la posición inicial del camino). */
19     + public void insertarEnemigoAlFinal(NodeEnemigo e) {
20         + NodeEnemigo nuevo = new NodeEnemigo(e);
21         + if (estaVacia()) {
22             + primero = ultimo = nuevo;
23         } else {
24             + nuevo.setAnterior(ultimo);
25             + ultimo.setSiguiente(nuevo);
26             + ultimo = nuevo;
27             + tamaño++;
28         }
29     }
30
31     + public void eliminarEnemigo(NodeEnemigo e) {
32         + if (estaVacia()) return;
33         + if (primero == e) primero = primero.getSiguiente();
34         + if (ultimo == e) ultimo = ultimo.getAnterior();
35         + e.setSiguiente(null);
36         + e.setAnterior(null);
37         + tamaño--;
38     }
39 }
```

Captura del commit 997921d en GitHub.



The screenshot shows the GitHub interface for commit 62f58a4. The file list on the left includes Scr.java and TableroPanel.java. The main area displays the code for Scr.java/TableroPanel.java, which is a Java class for the game board panel. The code includes package declarations, imports, and methods for drawing the game board, including the path, towers, and enemies.

```
1 import java.awt.BasicStroke;
2 import java.awt.Color;
3 import java.awt.Dimension;
4 import java.awt.Font;
5 import java.awt.GradientPaint;
6 import java.awt.Graphics;
7 import java.awt.Graphics2D;
8 import java.awt.RenderingHints;
9 import java.awt.event.MouseAdapter;
10 import java.awt.event.MouseEvent;
11 import java.awt.event.MouseMotionAdapter;
12 import java.swing.JPanel;
13
14 /**
15  * Dibuja el camino, las torres, los enemigos activos y los efectos del
16  * turno; también resuelve en qué posición del camino (o sobre qué torre)
17  * cae un clic del mouse. No contiene ninguna regla del juego; solo lee el
18  * estado que expone TowerDefensores y, ante un clic, se lo delega a ella.
19  */
20 + Las torres se pueden dibujar encima o debajo de la línea del camino según
21 + el lado donde se hizo clic para colocarla (ver @link TowerDefensores@setAdondeJ)).
22 +
23 public class TableroPanel extends JPanel {
24 }
```

Captura del commit 62f58a4 en GitHub.

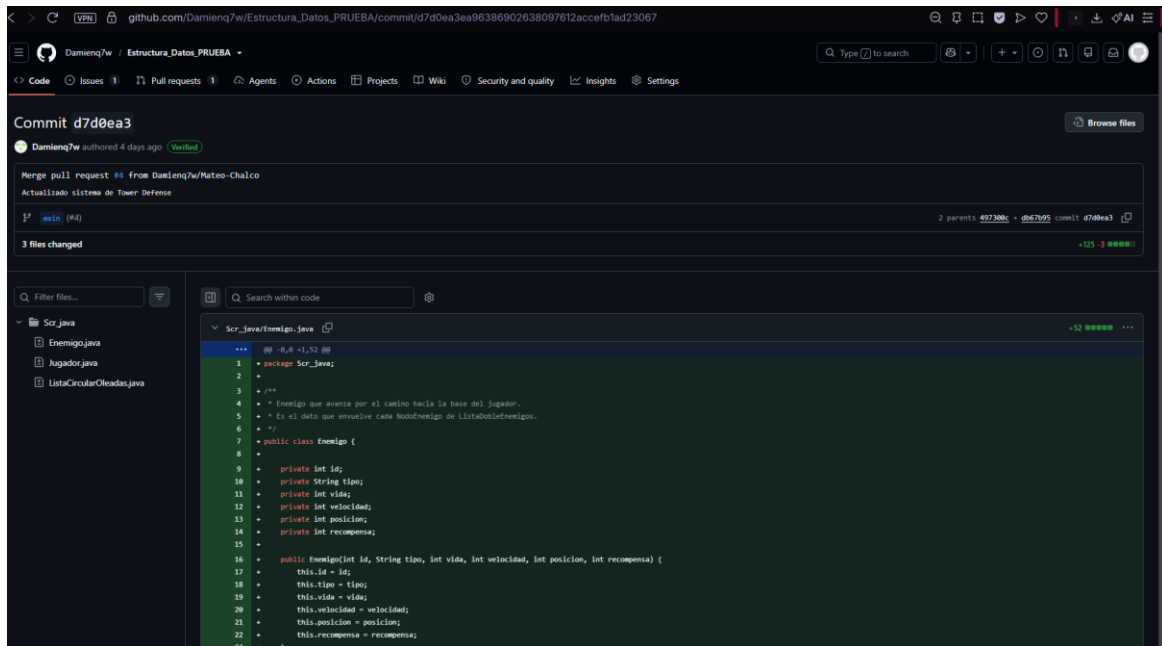
Chalco Tasna Kenneth Mateo

https://github.com/Damienq7w/Estructura_Datos_PRUEBA/commit/a6ffd54

a6ffd54 — "Actualizado sistema de Tower Defense". Creó `Enemigo.java` (avanzar, recibirDanio, estaDestruido), `Jugador.java` (vidas y derrota) y `ListaCircularOleadas.java` (registrar, avanzar y reiniciar el ciclo de oleadas).

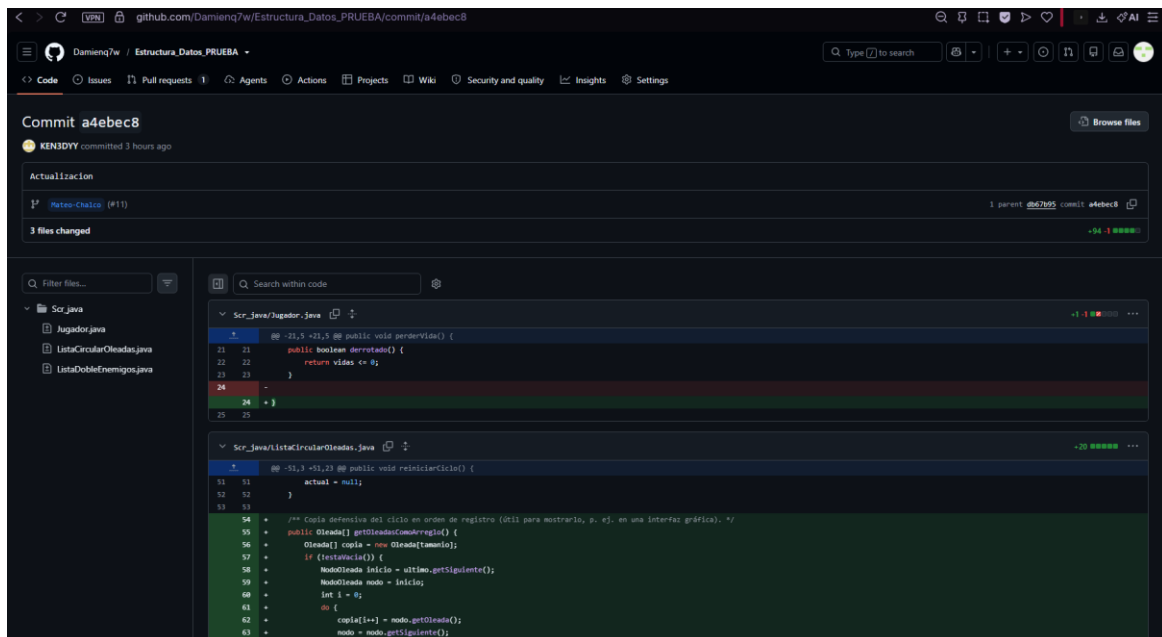
https://github.com/Damienq7w/Estructura_Datos_PRUEBA/commit/a4ebec8

a4ebec8 — "Actualizacion". Implementó `ListaDobleEnemigos.java` (inserción, búsqueda por id, eliminación y recorrido en ambos sentidos) y agregó `getOleadasComoArreglo()` y `getOleadaActual()` a `ListaCircularOleadas`.



The screenshot shows the GitHub interface for commit `d7d0ea3` by user `Damienq7w`. The commit message is "Actualizado sistema de Tower Defense". It shows a merge of pull request #4. The file list on the left includes `Enemigo.java`, `Jugador.java`, and `ListaCircularOleadas.java`. The main area displays the code for `Scr_java/Enemigo.java`, which defines a `Enemigo` class with attributes like `id`, `tipo`, `vidas`, `velocidad`, `posicion`, and `recompensa`. The code includes a constructor and a `public Enemigo` method.

Captura del commit a6ffd54 (fusionado como d7d0ea3, PR #4) en GitHub.



The screenshot shows the GitHub interface for commit `a4ebec8` by user `KENIDYY`. The commit message is "Actualizacion". It shows a merge of pull request #11. The file list on the left includes `Jugador.java`, `ListaCircularOleadas.java`, and `ListaDobleEnemigos.java`. The main area displays the code for `Scr_java/Jugador.java` and `Scr_java/ListaCircularOleadas.java`. The `Jugador.java` code includes a `public void perderVidas()` method. The `ListaCircularOleadas.java` code includes a `public void reiniciarCiclo()` method and a `public Oleada[] getOleadasComoArreglo()` method.

Captura del commit a4ebec8 en GitHub.

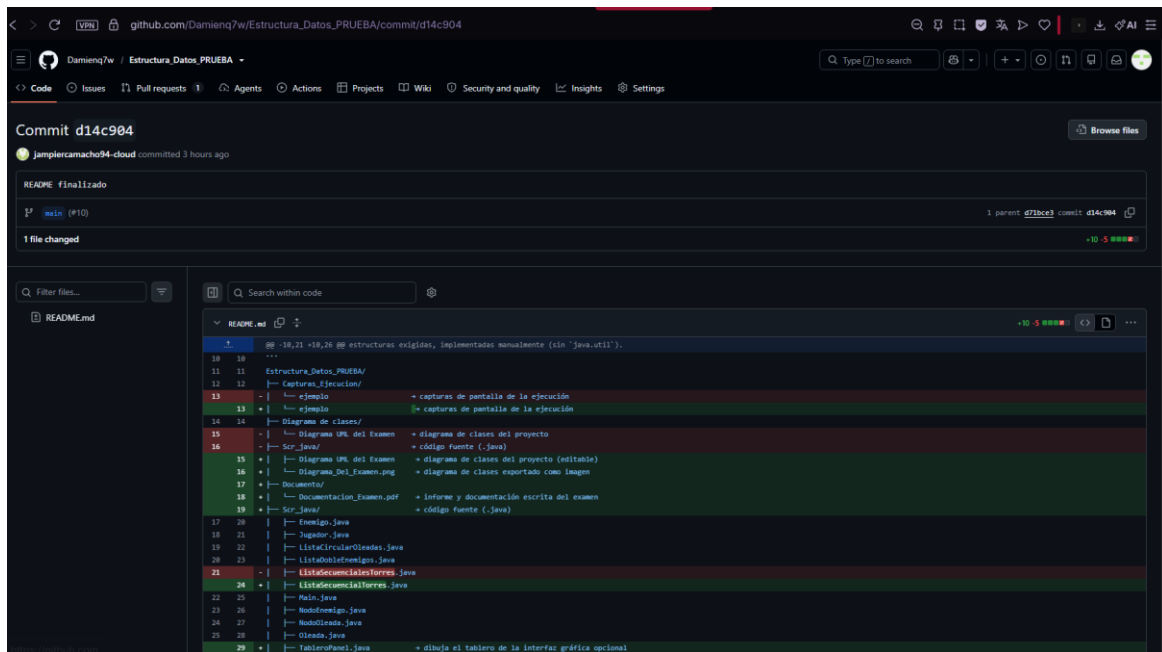
Camacho Monta Josue Jampier

https://github.com/Damieng7w/Estructura_Datos_PRUEBA/commit/d14c904

d14c904 — "README finalizado". Completó la documentación del README.md: estructura del repositorio, corrección del nombre de ListaSecuencialTorres y descripción de TableroPanel y TowerDefenseGUI.

https://github.com/Damieng7w/Estructura_Datos_PRUEBA/commit/4bfafa0

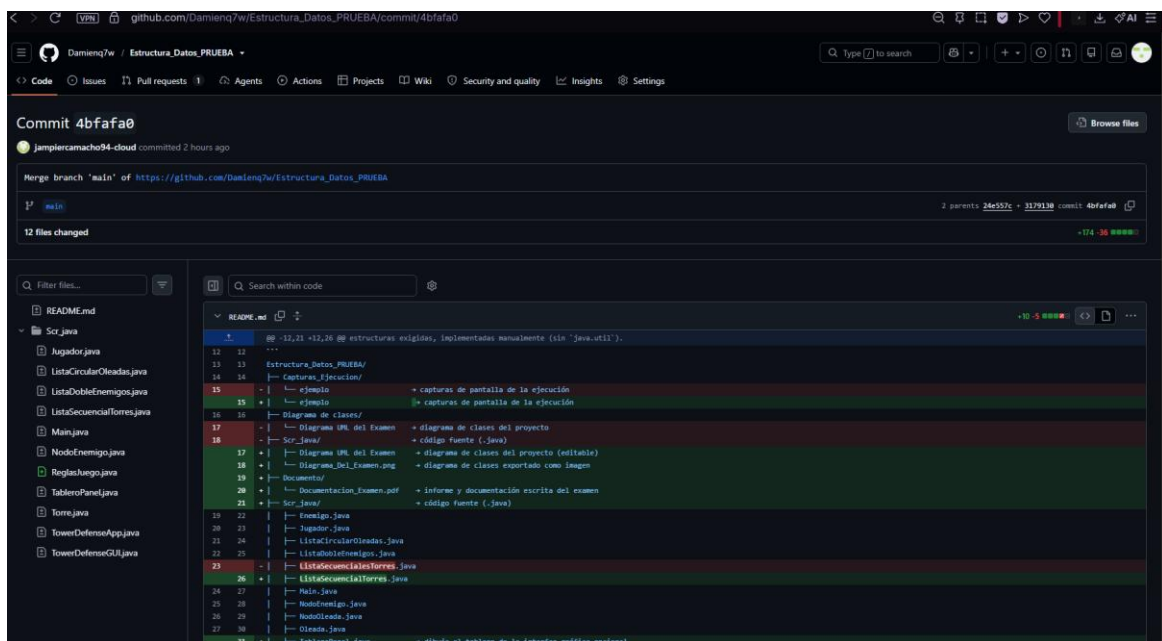
4bfafa0 — "Merge branch 'main'". Sincronizó las reglas de juego entre la consola y la interfaz gráfica creando ReglasJuego.java, y actualizó la documentación correspondiente (12 archivos, +174/-36).



The screenshot shows the GitHub interface for commit d14c904. The commit message is "README finalizado". The diff view shows changes to the README.md file. The file content is as follows:

```
@@ -10,21 +10,26 @@ estructuras exigidas, implementadas manualmente (sin 'java.util').
...
10 10  ...
11 11  Estructura_Datos_PRUEBA/
12 12  |--- Capturas_fjecucion/
13 13  |   |--- ejemplo
13 13  |   |--- ejemplo
13 13  |   |--- ejemplo
14 14  |--- Diagrama de clases/
15 15  |   |--- Diagrama UML del examen
15 15  |   |--- Diagrama UML del examen
16 16  |   |--- Diagrama UML del examen
16 16  |   |--- Diagrama UML del examen
16 16  |   |--- Diagrama UML del examen
17 17  |--- Diagrama UML del examen
18 18  |--- Diagrama UML del examen
19 19  |--- Diagrama UML del examen
20 20  |--- Diagrama UML del examen
21 21  |--- Diagrama UML del examen
22 22  |--- Diagrama UML del examen
23 23  |--- Diagrama UML del examen
24 24  |--- Diagrama UML del examen
25 25  |--- Diagrama UML del examen
26 26  |--- Diagrama UML del examen
27 27  |--- Diagrama UML del examen
28 28  |--- Diagrama UML del examen
29 29  |--- Diagrama UML del examen
30 30  |--- Diagrama UML del examen
31 31  |--- Diagrama UML del examen
32 32  |--- Diagrama UML del examen
33 33  |--- Diagrama UML del examen
34 34  |--- Diagrama UML del examen
35 35  |--- Diagrama UML del examen
36 36  |--- Diagrama UML del examen
37 37  |--- Diagrama UML del examen
38 38  |--- Diagrama UML del examen
39 39  |--- Diagrama UML del examen
40 40  |--- Diagrama UML del examen
41 41  |--- Diagrama UML del examen
42 42  |--- Diagrama UML del examen
43 43  |--- Diagrama UML del examen
44 44  |--- Diagrama UML del examen
45 45  |--- Diagrama UML del examen
46 46  |--- Diagrama UML del examen
47 47  |--- Diagrama UML del examen
48 48  |--- Diagrama UML del examen
49 49  |--- Diagrama UML del examen
50 50  |--- Diagrama UML del examen
51 51  |--- Diagrama UML del examen
52 52  |--- Diagrama UML del examen
53 53  |--- Diagrama UML del examen
54 54  |--- Diagrama UML del examen
55 55  |--- Diagrama UML del examen
56 56  |--- Diagrama UML del examen
57 57  |--- Diagrama UML del examen
58 58  |--- Diagrama UML del examen
59 59  |--- Diagrama UML del examen
60 60  |--- Diagrama UML del examen
61 61  |--- Diagrama UML del examen
62 62  |--- Diagrama UML del examen
63 63  |--- Diagrama UML del examen
64 64  |--- Diagrama UML del examen
65 65  |--- Diagrama UML del examen
66 66  |--- Diagrama UML del examen
67 67  |--- Diagrama UML del examen
68 68  |--- Diagrama UML del examen
69 69  |--- Diagrama UML del examen
70 70  |--- Diagrama UML del examen
71 71  |--- Diagrama UML del examen
72 72  |--- Diagrama UML del examen
73 73  |--- Diagrama UML del examen
74 74  |--- Diagrama UML del examen
75 75  |--- Diagrama UML del examen
76 76  |--- Diagrama UML del examen
77 77  |--- Diagrama UML del examen
78 78  |--- Diagrama UML del examen
79 79  |--- Diagrama UML del examen
80 80  |--- Diagrama UML del examen
81 81  |--- Diagrama UML del examen
82 82  |--- Diagrama UML del examen
83 83  |--- Diagrama UML del examen
84 84  |--- Diagrama UML del examen
85 85  |--- Diagrama UML del examen
86 86  |--- Diagrama UML del examen
87 87  |--- Diagrama UML del examen
88 88  |--- Diagrama UML del examen
89 89  |--- Diagrama UML del examen
90 90  |--- Diagrama UML del examen
91 91  |--- Diagrama UML del examen
92 92  |--- Diagrama UML del examen
93 93  |--- Diagrama UML del examen
94 94  |--- Diagrama UML del examen
95 95  |--- Diagrama UML del examen
96 96  |--- Diagrama UML del examen
97 97  |--- Diagrama UML del examen
98 98  |--- Diagrama UML del examen
99 99  |--- Diagrama UML del examen
100 100 |--- Diagrama UML del examen
```

Captura del commit d14c904 en GitHub.



The screenshot shows the GitHub interface for commit 4bfafa0. The commit message is "Merge branch 'main' of https://github.com/Damieng7w/Estructura_Datos_PRUEBA". The diff view shows changes to the README.md file. The file content is as follows:

```
@@ -10,21 +10,26 @@ estructuras exigidas, implementadas manualmente (sin 'java.util').
...
10 10  ...
11 11  Estructura_Datos_PRUEBA/
12 12  |--- Capturas_fjecucion/
13 13  |   |--- ejemplo
13 13  |   |--- ejemplo
13 13  |   |--- ejemplo
14 14  |--- Diagrama de clases/
15 15  |   |--- Diagrama UML del examen
15 15  |   |--- Diagrama UML del examen
16 16  |   |--- Diagrama UML del examen
16 16  |   |--- Diagrama UML del examen
16 16  |   |--- Diagrama UML del examen
17 17  |--- Diagrama UML del examen
18 18  |--- Diagrama UML del examen
19 19  |--- Diagrama UML del examen
20 20  |--- Diagrama UML del examen
21 21  |--- Diagrama UML del examen
22 22  |--- Diagrama UML del examen
23 23  |--- Diagrama UML del examen
24 24  |--- Diagrama UML del examen
25 25  |--- Diagrama UML del examen
26 26  |--- Diagrama UML del examen
27 27  |--- Diagrama UML del examen
28 28  |--- Diagrama UML del examen
29 29  |--- Diagrama UML del examen
30 30  |--- Diagrama UML del examen
31 31  |--- Diagrama UML del examen
32 32  |--- Diagrama UML del examen
33 33  |--- Diagrama UML del examen
34 34  |--- Diagrama UML del examen
35 35  |--- Diagrama UML del examen
36 36  |--- Diagrama UML del examen
37 37  |--- Diagrama UML del examen
38 38  |--- Diagrama UML del examen
39 39  |--- Diagrama UML del examen
40 40  |--- Diagrama UML del examen
41 41  |--- Diagrama UML del examen
42 42  |--- Diagrama UML del examen
43 43  |--- Diagrama UML del examen
44 44  |--- Diagrama UML del examen
45 45  |--- Diagrama UML del examen
46 46  |--- Diagrama UML del examen
47 47  |--- Diagrama UML del examen
48 48  |--- Diagrama UML del examen
49 49  |--- Diagrama UML del examen
50 50  |--- Diagrama UML del examen
51 51  |--- Diagrama UML del examen
52 52  |--- Diagrama UML del examen
53 53  |--- Diagrama UML del examen
54 54  |--- Diagrama UML del examen
55 55  |--- Diagrama UML del examen
56 56  |--- Diagrama UML del examen
57 57  |--- Diagrama UML del examen
58 58  |--- Diagrama UML del examen
59 59  |--- Diagrama UML del examen
60 60  |--- Diagrama UML del examen
61 61  |--- Diagrama UML del examen
62 62  |--- Diagrama UML del examen
63 63  |--- Diagrama UML del examen
64 64  |--- Diagrama UML del examen
65 65  |--- Diagrama UML del examen
66 66  |--- Diagrama UML del examen
67 67  |--- Diagrama UML del examen
68 68  |--- Diagrama UML del examen
69 69  |--- Diagrama UML del examen
70 70  |--- Diagrama UML del examen
71 71  |--- Diagrama UML del examen
72 72  |--- Diagrama UML del examen
73 73  |--- Diagrama UML del examen
74 74  |--- Diagrama UML del examen
75 75  |--- Diagrama UML del examen
76 76  |--- Diagrama UML del examen
77 77  |--- Diagrama UML del examen
78 78  |--- Diagrama UML del examen
79 79  |--- Diagrama UML del examen
80 80  |--- Diagrama UML del examen
81 81  |--- Diagrama UML del examen
82 82  |--- Diagrama UML del examen
83 83  |--- Diagrama UML del examen
84 84  |--- Diagrama UML del examen
85 85  |--- Diagrama UML del examen
86 86  |--- Diagrama UML del examen
87 87  |--- Diagrama UML del examen
88 88  |--- Diagrama UML del examen
89 89  |--- Diagrama UML del examen
90 90  |--- Diagrama UML del examen
91 91  |--- Diagrama UML del examen
92 92  |--- Diagrama UML del examen
93 93  |--- Diagrama UML del examen
94 94  |--- Diagrama UML del examen
95 95  |--- Diagrama UML del examen
96 96  |--- Diagrama UML del examen
97 97  |--- Diagrama UML del examen
98 98  |--- Diagrama UML del examen
99 99  |--- Diagrama UML del examen
100 100 |--- Diagrama UML del examen
```

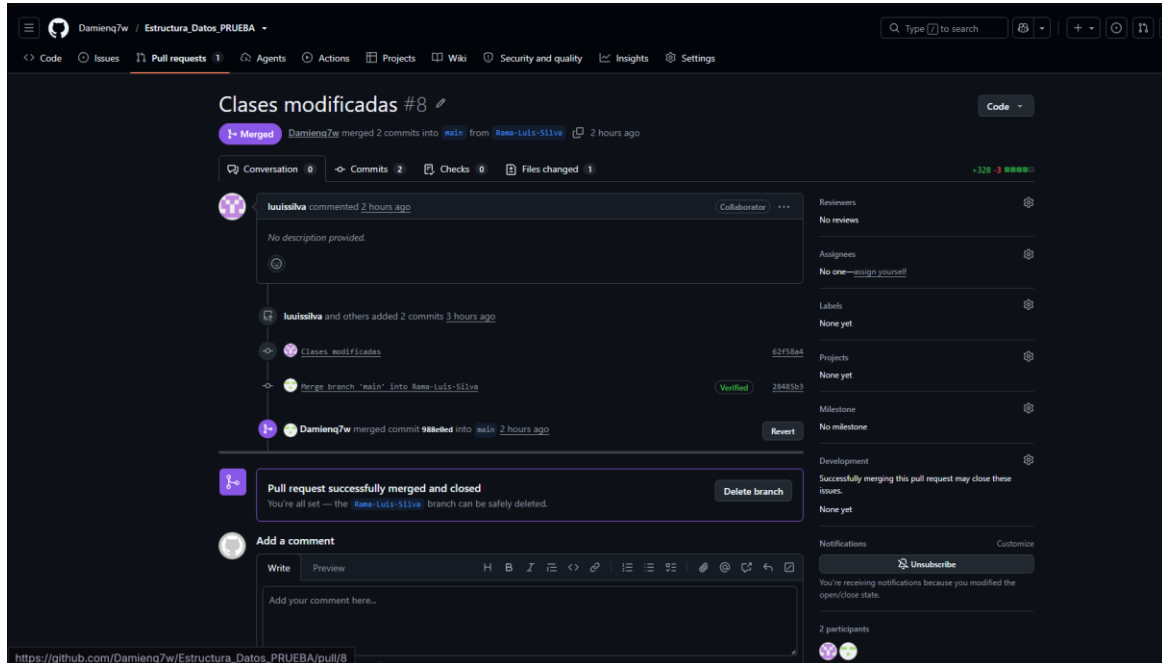
Captura del commit 4bfafa0 en GitHub.

Tacuri Santillan Monica Sara: no registra commits en el repositorio, ya que no llegó a tiempo a la prueba práctica y no realizó la parte que tenía asignada.

4. Pull request integrado

https://github.com/Damienq7w/Estructura_Datos_PRUEBA/pull/8

"Clases modificadas" (PR #8): creado por Silva Camuendo Luis Alexander desde la rama Rama-Luis-Silva (commits 62f58a4 y 28485b3), y fusionado a main por Cunalata Mendoza Damian Alexander. Incorpora la clase TableroPanel completa (+328 líneas) tras revisar los cambios antes de integrarlos.

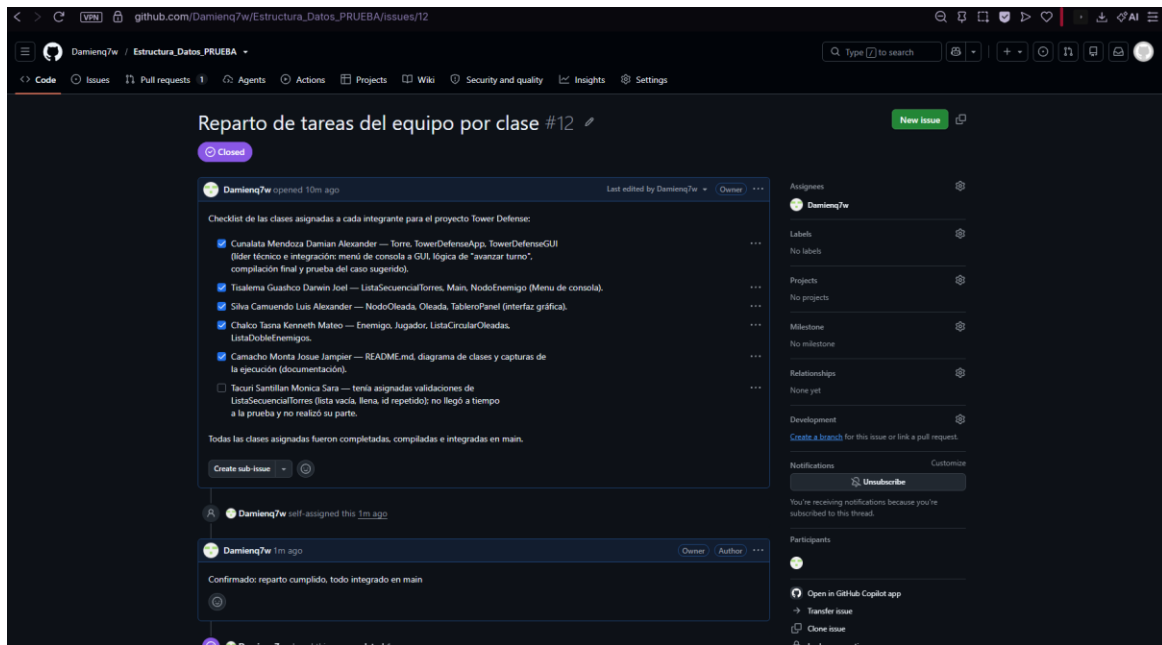


Captura del PR #8 fusionado en GitHub.

5. Issue asignada y cerrada

https://github.com/Damienq7w/Estructura_Datos_PRUEBA/issues/12

"Reparto de tareas del equipo por clase" (issue #12): checklist con la clase o clases asignadas a cada integrante. Fue asignada a Cunalata Mendoza Damian Alexander y cerrada tras confirmar que todas las clases asignadas fueron completadas, compiladas e integradas en main.



Captura de la issue #12 asignada y cerrada en GitHub.

6. Reflexión sobre el trabajo colaborativo

Yo implementé Enemigo, Jugador, ListaCircularOleadas y ListaDobleEnemigos, cubriendo la lógica de combate y dos de las tres estructuras exigidas. Dividimos las tareas según las clases y estructuras que pedía el enunciado, y cada quien subió su parte a su propia rama en GitHub, integrada luego a main mediante Pull Requests. Decidimos ese reparto desde la primera reunión para que cada estructura avanzara casi independiente antes de unirla en TowerDefenseApp. La principal dificultad fue coordinar nombres de clases y métodos entre ramas distintas, lo que resolvimos revisando el código antes de cada fusión. Verificamos el aporte de cada integrante revisando su historial de commits y probando el juego completo, en consola y en la interfaz gráfica, antes de entregar. Mónica tenía asignadas las validaciones de ListaSecuencialTorres, pero no llegó a tiempo a la prueba práctica y no logró completar su parte.